서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

# 실험 후 보고서

## STM32 인터럽트·타이머·PWM 및 FPGA 개별 실습

| | |
|---|---|
| 학과 | 전자전기컴퓨터공학부 |
| 이름 / 학번 | 조성현 / 2023440129 |
| 작성일 | 2026년 10월 19일 |
| 대상 | 버튼 인터럽트 · 주기 타이머 · PWM 제공 예제 · STM32 과제3 · STM32 과제4 · FPGA 피아노 · 신호등 · ADC 실습 |
| 검증환경 | Windows 11 / VS Code / STM32 CMake / VaporView / AMD Vivado 2026.1 XSim |

**소스 식별**

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab5

제출 태그: lab5-post-v1

커밋 해시: 518ceffa7a654b82bb0a8e11fcc536b6798fddbd

## 1. 실험 목적

본 실험에서는 STM32의 외부 인터럽트와 타이머를 사용하여 버튼 입력에 따라 LED의 상태 변경 간격을 제어하고, PWM 출력의 듀티비를 변경한다. FPGA에서는 피아노의 음 선택 및 구형파 생성, 신호등의 방향별 신호 제어, ADC와의 직렬 송수신을 구현한다. 각 설계에 대해 코드와 시뮬레이션 결과를 확인하고, Vivado 구현 결과와 실제 보드 시연을 비교한다. 이를 통해 STM32 및 FPGA 보드에서 다양한 회로를 구현하고, 동작원리를 이해하는 것에 초점을 둔다.

## 2. 관련 이론

### 폴링과 인터럽트의 응답 방식

폴링은 프로그램이 반복문에서 버튼의 상태를 계속 읽어 변화가 생겼는지 확인하는 방식이다. LAB4에서는 메인 루프가 버튼 상태와 경과 시간을 확인하여 LED 동작을 제어하였다. 버튼이 검사 직후에 눌리면 다음 검사 차례까지 발견되지 않으므로, 응답 시간은 프로그램이 버튼을 얼마나 자주 검사하는지에 영향을 받는다. 일정한 간격 $\Delta t$마다 검사하는 주기적 폴링이라면 검사까지 기다리는 시간은 최대 약 $\Delta t$이다. 다만 LAB4의 메인 루프에 일정한 검사 주기가 설정된 것은 아니므로, 이 수치를 LAB4의 실제 응답 시간으로 해석해서는 안 된다.[1]

인터럽트 방식에서는 버튼 입력이 발생하면 하드웨어가 처리 요청을 전달하고, 프로세서가 해당 인터럽트 서비스 루틴(ISR)을 실행한다. 이번 과제 3은 버튼 입력을 외부 인터럽트로 처리하고 TIM6 인터럽트로 LED의 점멸 시점을 관리한다. 따라서 버튼 처리가 메인 루프의 다음 검사 차례를 기다리지 않는다는 점에서 LAB4의 폴링 방식과 다르다. 그러나 인터럽트에도 진입 및 처리 시간이 들기 때문에 "인터럽트가 언제나 폴링보다 빠르다"고 단정할 수는 없다.[1]

## 3. 실험 환경 및 검증 방법

### 사용 장비와 연결

| 항목 | 설정 및 용도 |
|---|---|
| STM32 보드 | NUCLEO-G474RE |
| 공통 입력 | PC13 사용자 버튼 |
| 과제3 출력 | PA5 내장 LED, TIM6 주기 인터럽트 |
| 과제4 출력 | TIM3 채널 1 PWM, PB4, 오실로스코프 |
| STM32 개발 | VS Code에서 과제별 빌드 및 보드 실행 |
| FPGA 개발 | AMD Vivado 2026.1, XSim, Hardware Manager |
| FPGA 보드 | Combo II-DLD S75, xc7s75fgga484-1 |
| 결과 관리 | Git 및 GitHub |

### 공통 검증 순서

STM32는 과제3과 과제4의 코드를 각각 빌드하고 보드에 실행한 뒤 버튼 입력과 LED 또는 PWM 출력을 확인하였다. FPGA는 각 테스트벤치에서 입력과 예상 출력을 비교하고, 파형을 확인한 다음 설계 모듈을 최상위로 설정하여 합성·구현·비트스트림 생성을 진행하였다. 이후 Hardware Manager에서 보드를 프로그램하고 사진과 영상으로 동작을 기록하였다.

### 실험별 설계파일

| 항목 | 설정 |
|---|---|
| 피아노 | piano.v, note_select.v, square_tone.v, tb.sv, piano.xdc |
| 신호등 | traffic.v, second_tick.v, traffic_sequence.v, traffic_lights.v, tb.sv, traffic.xdc |
| ADC | adc.v, adc_command.v, serial_frame16.v, tb.sv, adc.xdc |

**링크:** 실습1 설계 및 검증 파일

**링크:** 실습2 설계 및 검증 파일

**링크:** 실습3 설계 및 검증 파일

## 4. STM32 과제3: LED 상태 변경 간격 제어

### 설계 및 예상 동작

사용자 버튼을 한 번 누를 때마다 PA5 내장 LED의 상태 변경 간격이 1초 → 2초 → 4초 → 8초 → 1초로 순환하도록 설계하였다. 여기서 1초는 LED가 켜진 상태에서 꺼지거나, 꺼진 상태에서 켜지기 까지의 간격이다. 버튼을 누르지 않는 동안에도 LED는 현재 선택된 간격으로 계속 상태를 변경한다.

| 상태 | TIM6 100ms 인터럽트 횟수 | 예상 LED 동작 |
|---|---|---|
| 시작 | 10회 | 1초마다 상태 변경 |
| 버튼 1회 | 20회 | 2초마다 상태 변경 |
| 버튼 2회 | 40회 | 4초마다 상태 변경 |
| 버튼 3회 | 80회 | 8초마다 상태 변경 |
| 버튼 4회 | 10회 | 다시 1초마다 상태 변경 |

### 코드 동작

과제3 코드에서는 TIM6을 인터럽트 방식으로 시작하고,

인터럽트가 발생할 때마다 elapsed_ticks를 증가시킨다. 이 값이 interval_ticks에 도달하면 HAL_GPIO_TogglePin()으로 PA5 LED를 반전하고 누적 횟수를 초기화한다. 버튼 인터럽트에서는 200 ms 이내의 중복 입력을 제외한 뒤 interval_ticks를 10·20·40·80 순서로 변경한다. 간격이 바뀌는 시점에는 경과 횟수와 TIM6 카운터도 초기화한다.

### 빌드 및 보드 결과

과제3 코드를 빌드한 뒤 보드의 버튼을 사용하여 PA5 LED의 동작을 시연하였다. 그림 1은 LED 보드 시연 사진이다. 정지 사진에는 한 시점의 LED 상태가 나타나므로, 버튼을 누를 때마다 상태 변경 간격이 순환하는 과정은 연결한 시연영상으로 제시한다.

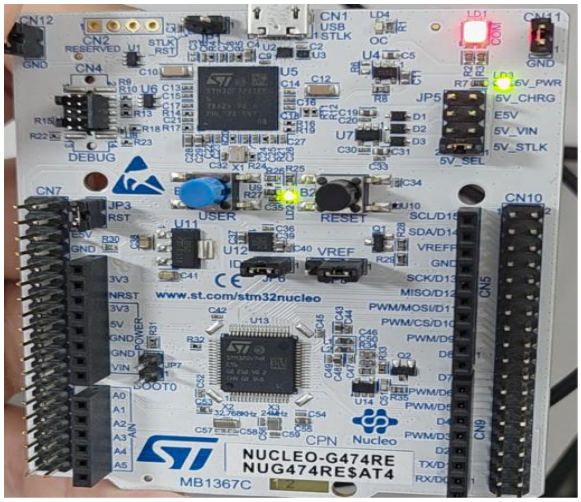



**그림 1: 과제3 PA5 내장 LED 보드 시연 사진**

**링크:** [과제3 보드 시연 영상]

## 5. STM32 과제4: PWM 듀티비 변경

### 설계 변경

TIM3 채널 1을 사용하여 PB4에서 약 1 kHz의 PWM 파형을 출력한다. PC13 버튼을 누를 때마다 듀티비가 25% → 50% → 75% → 25%로 순환한다. PWM 주기가 1 ms라면 예상 HIGH 시간은 다음과 같다.

| 상태 | 듀티비 | TIM3 비교값 |
|---|---|---|
| 시작 | 25% | 250 |
| 버튼 1회 | 50% | 500 |
| 버튼 2회 | 75% | 750 |
| 버튼 3회 | 25% | 250 |

### 코드 동작

duty_percent는 처음에 25로 설정된다. 초기화 함수가 TIM3 채널 1의 비교값을 duty_percent × 10으로 설정하고 PWM을 시작한다. 버튼 인터럽트에서는 200 ms 이내의 중복 입력을 제외하고 듀티비를 25씩 증가시키며, 75 다음에는 25로 되돌린다. __HAL_TIM_SET_COMPARE()가 변경된 비교값을 반영한다. TIM3의 ARR은 999로 유지되므로 버튼 입력은 주기보다 HIGH 구간의 비율을 바꾼다.

### 빌드 및 보드 결과

과제 4를 빌드하고 GND핀은 GND 단자와 연결하고 PB4 출력은 PWM/D5단자에 연결하여 파형을 오실로스코프로 확인하였다. 그림 2에서는 반복되는 PWM 파형을 볼 수 있다. 버튼을 눌렀을 때 듀티비가 순환하는 과정은 연결한 시연영상으로 제시한다.

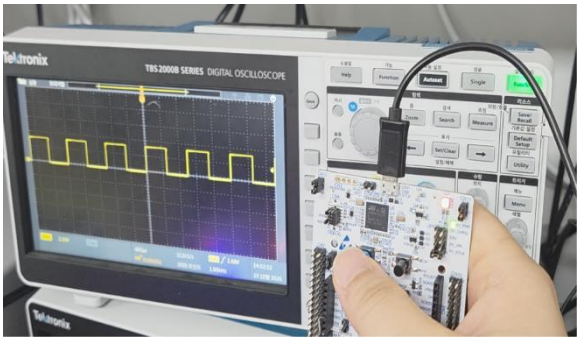



**그림 2: 과제4 PB4 PWM 오실로스코프 시연 사진**

**링크:** [과제4 PWM 시연 영상]

## 6. FPGA 실습1: 피아노

### 회로 구성

note_select는 하나의 키가 눌렸을 때 해당 음의 주파수를 선택한다. square_tone은 선택된 주파수에 맞게 출력 piezo를 주기적으로 반전시켜 구형파를 만든다. 키를 누르지 않거나 둘 이상 동시에 누르면 주파수 선택값이 0이 되어 출력을 멈춘다.

| 키 입력 | 선택주파수 | 키 입력 | 선택주파수 |
|---|---|---|---|
| 8'h80 | 262Hz | 8'h08 | 392Hz |
| 8'h40 | 294Hz | 8'h04 | 440Hz |
| 8'h20 | 330Hz | 8'h02 | 494Hz |
| 8'h10 | 349Hz | 8'h01 | 523Hz |

### Vivado 실험사진

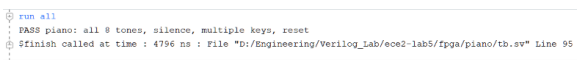



**그림 3: 실습1 Vivado 시뮬레이션 PASS**

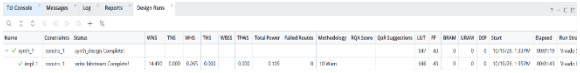
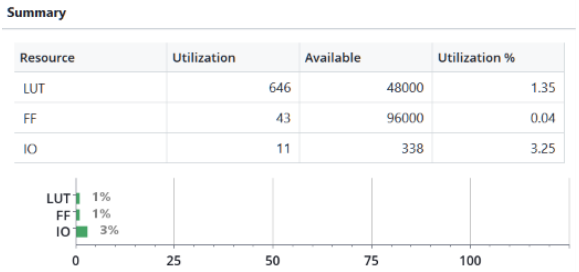


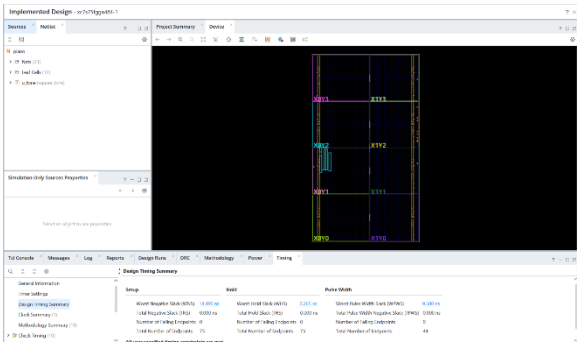
**그림 4: 실습1 합성 및 구현 완료 결과**




그 림 5 : 실 습 1 F P G A 자 원 사 용 량



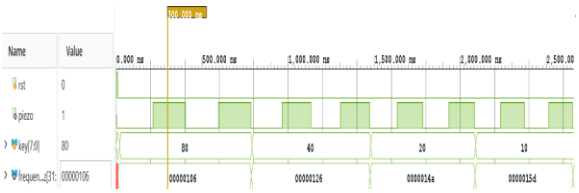
그 림 6 : 실 습 1 D R C 검 사 결 과



그 림 7 : 실 습 1 타 이 밍 분 석 결 과



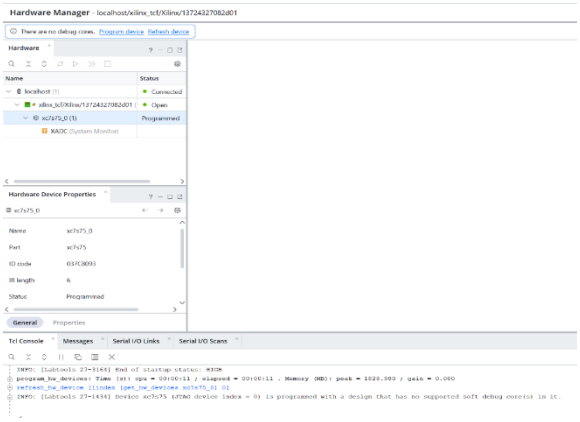
그 림 8 : 실 습 1 B i t H a s h 확 인




그 림 9 : 실 습 1 입 력 조 합 에 따 른 출 력 파 형

### Vivado 시뮬레이션 및 파형 비교

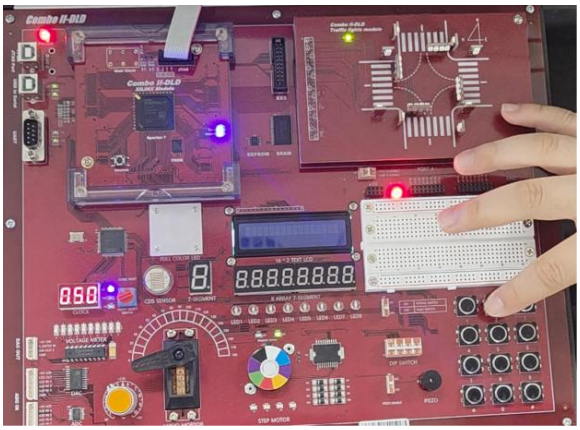
테스트벤치는 여덟 개의 단일 키 입력에 대해 선택 주파수와 구형파 반전 시점을 확인하고, 동시 입력·무입력의 무음 처리와 리셋도 검사한다. 그림 9의 파형에서는 키가 8'h80 → 8'h40 → 8'h20 → 8'h10으로 변할 때 주파수 선택 값이 차례로 262 → 294 → 330 → 349 Hz에 해당하는 값으로 변한다. 화면에 표시된 00000106, 00000126, 0000014a, 0000015d는 이에 해당하는 16진수 표시 값이다. 즉 시뮬레이션 파형이 설계한 회로의 동작과 정확하게 일치하는 결과를 보이고 있음을 알 수 있다.




그 림 10 : 실 습 1 P r o g r a m D e v i c e 확 인

### FPGA 보드 시연

그림 11의 보드 시연 사진은 피아노 입력 버튼에 따라 피에조 출력이 발생하는 것을 나타낸다. 버튼을 1번부터 순서대로 누르면 주파수가 점차 증가하고, 그에 따라 소리의 음이 높아지는 것을 확인할 수 있다. 키에 따른 소리의 변화는 실습 1 보드 시연 영상에서 확인할 수 있다.



그 림 11 : 실 습 1 F P G A 보 드 시 연 사 진

링크: FPGA 실습1 보드 시연 영상

## 7. FPGA 실습2: 신호등

### 회로 구성

second_tick은 시간 진행용 tick과 점멸 신호 blink를 만들고, traffic_sequence는 선택 방향과 phase를 갱신한다. traffic_lights는 이에 따라 차량과 보행자 LED를 정한다. 한 방향에서 phase 0~12를 마치면 다음 방향으로 넘어가며, 네 방향이 순환한다.

| Phase | 선택 방향 차량 신호 | 대응 보행자 신호 |
|---|---|---|
| 0~3 | 초록 | 초록 유지 |
| 4~10 | 초록 | 초록 점멸 |
| 11~12 | 노랑 | 빨강 |

| | | |
|---|---|---|
| 12 다음 | 다음 방향으로 변경 | 다음 방향의 신호 시작 |

**Vivado 실험사진**

```
run all
PASS traffic: full cycle, pedestrian blink, flicker, reset
$finish called at time : 2186 ns : File "D:/Engineering/Verilog_Lab/ece2-lab5/fpga/traffic/tb.sv" Line 107
```

그림 12: 실습2 Vivado 시뮬레이션 PASS

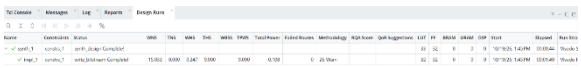


그림 13: 실습2 합성 및 구현 완료 결과

Summary

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 32 | 48000 | 0.07 |
| FF | 32 | 96000 | 0.03 |
| IO | 27 | 338 | 7.99 |

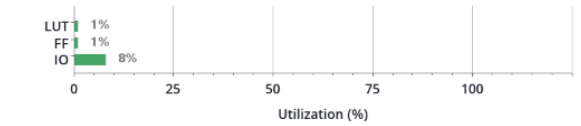



그림 14: 실습2 FPGA 자원 사용량

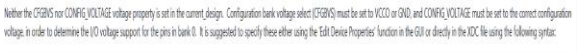


그림 15: 실습2 DRC 검사 결과

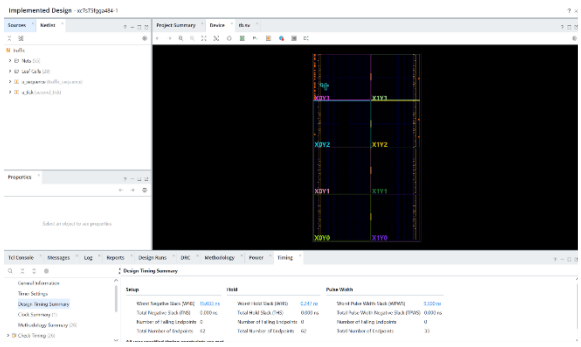


그림 16: 실습2 타이밍 분석 결과

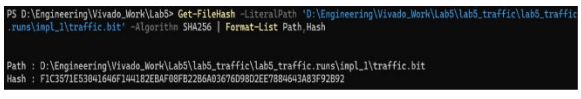



그림 17: 실습2 Bit Hash 확인

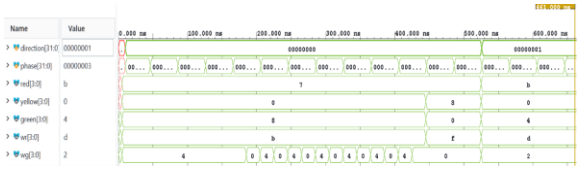


그림 18: 실습2 입력 조합에 따른 출력 파형

**Vivado 시뮬레이션 및 파형 비교**

테스트벤치에서는 클럭을 생성하고, 신호등의 네 방향에 대해 각 13단계의 출력이 예상값과 일치하는지 검사하였다. 보행자 신호의 점멸, flicker 입력에 따른 점멸 모드, 리셋 후 초기 상태 복귀도 검사하였다. Vivado 시뮬레이션은 PASS traffic: full cycle, pedestrian blink, flicker, reset을 출력하였다.

Verilog 파형과 Vivado 파형 모두에서 direction=0인 동안 phase가 증가하고, phase=10까지 차량 초록 신호가 유지되다가 phase=11에서 노랑 신호로 전환되는 것을 확인하였다. phase=12 다음에는 direction이 0에서 1로 바뀌고 phase가 0으로 돌아가며, 다음 방향의 차량 및 보행자 신호가 시작된다. 보행자 초록 신호는 중간 단계에서 점멸하다가 차량 노랑 단계에서 꺼진다. 두 파형은 이 전환 순서가 일치한다.

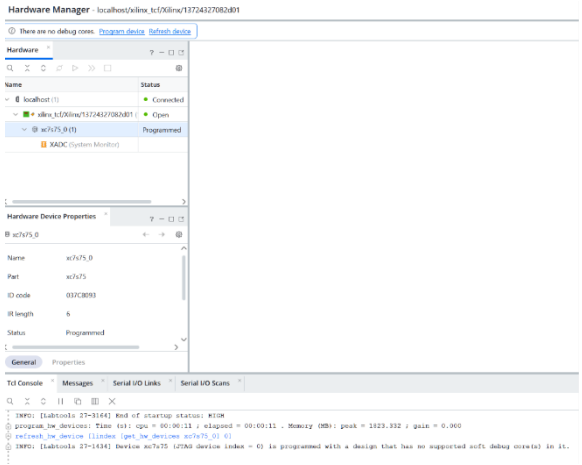



그림 19: 실습2 Program Device 확인

**FPGA 보드 시연**

그림 20의 보드 시연 사진은 신호등 확장 모듈의 한 시점에서 차량, 보행자 신호가 켜진 상태를 나타낸다. 정해진 주기에 따라 방향 전환이 발생하였으며, 두 번째 버튼을 눌렀을 때는 노란색 및 빨간색 점멸 신호가 지속적으로 출력되었다. 정확한 보드 시연은 아래 영상을 통해 확인할 수 있다.

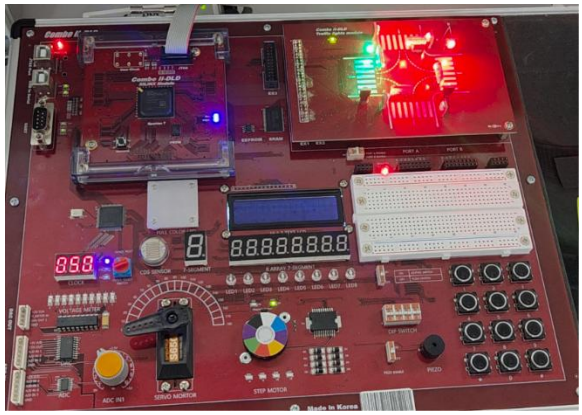


그림 20: 실습2 FPGA 보드 시연 사진

링크: FPGA 실습2 보드 시연 영상

## 8. FPGA 실습3: ADC 직렬 송수신

### 회로 구성

adc_command는 채널 정보를 담은 16비트 명령을 만들고, serial_frame16은 ADC의 칩 선택·직렬 클록·송수신 신호를 제어한다. adc는 수신값의 하위 12비트를 sample에 저장하고 sample[11:4]를 8개 LED에 출력한다. LED는 차례로 한 칸씩 이동하는 표시가 아니라 측정값의 8비트 이진 표시이다.

| 스위치 입력 | 확인한 입력 대상 | 예상 LED 반응 |
|---|---|---|
| 000 | 가변저항 | 가변저항을 조절하면 표시 값 변화 |
| 001 | 조도 센서 | 밝기를 바꾸면 표시 값 변화 |

### Vivado 실험사진

```
run all
PASS ADC: 3 serial frames, command bits, sample, reset
$finish called at time : 2016 ns : File "D:/Engineering/Verilog_Lab/ece2-lab5/fpga/adc/tb.sv" Line 69
```

그림 21: 실습3 Vivado 시뮬레이션 PASS

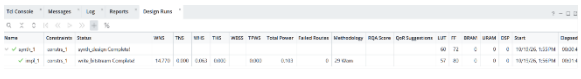


그림 22: 실습3 합성 및 구현 완료 결과

Summary

| Resource | Utilization | Available | Utilization % |
|---|---|---|---|
| LUT | 57 | 48000 | 0.12 |
| FF | 80 | 96000 | 0.08 |
| IO | 30 | 338 | 8.88 |

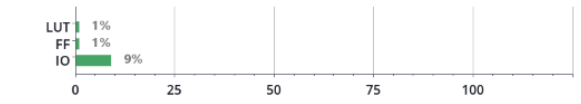



그림 23: 실습3 FPGA 자원 사용량




그림 24: 실습3 DRC 검사 결과

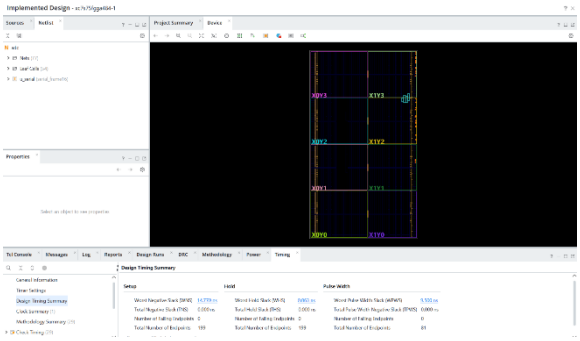


그림 25: 실습3 타이밍 분석 결과

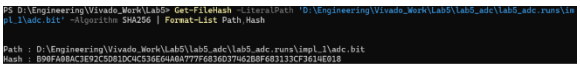



그림 26: 실습3 Bit Hash 확인

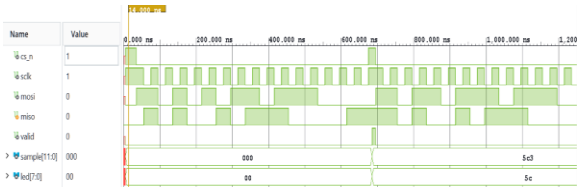



그림 27: 실습3 입력 조합에 따른 출력 파형

### Vivado 시뮬레이션 및 파형 비교

기존 테스트벤치는 ADC의 16비트 직렬 프레임 세 개를 검사하였다. 저장된 Verilog 파형과 Vivado 파형에서는 CS가 활성화된 동안 SCLK에 맞춰 송수신 신호가 변하고, 첫 프레임의 수신이 끝난 뒤 data_valid가 발생하는 것을 확인할 수 있다. 테스트벤치의 응답값 16'hA5C3에서 하위 12비트인 12'h5C3이 sample에 저장되고, 그 상위 8비트인 8'h5C가 LED 출력에 나타난다. 저장된 Vivado 실행 결과는 세 프레임 검사 후 PASS ADC: 3 serial frames, command bits, sample, reset을 출력하였다. 두 저장된 파형에서 이 첫 프레임의 출력 결과가 일치한다.

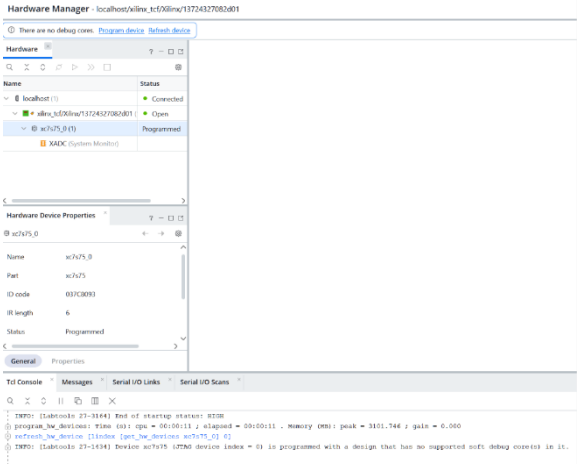



그림 28: 실습3 Program Device 확인

### FPGA 보드 시연

그림29의 보드 시연 사진은 특정 아날로그 입력에 대해 디지털 출력으로 변환되는 동작을 보여준다. 보드에서 DIC 스위치가 000인 상태에서 가변저항의 값을 변경하면 LED의 출력 값이 변하는 것을 확인할 수 있다. 정확히는 가변 전압 값을 올려 LED 출력 값이 증가하는 것을 확인하였다. 그리고, DIC 스위치가 001인 상태에서 조도 센서에서 감지되는 빛의 양에 따라 LED 출력 값이 변하는 것을 확인할 수 있다. 정확히는 빛의 많이 감지될수록 LED 출력 값이 증가하는 것을 확인하였다. 각각의 스위치 상태에서 서로의 입력이 독립적으로 작용하는 것도 검증하였다.

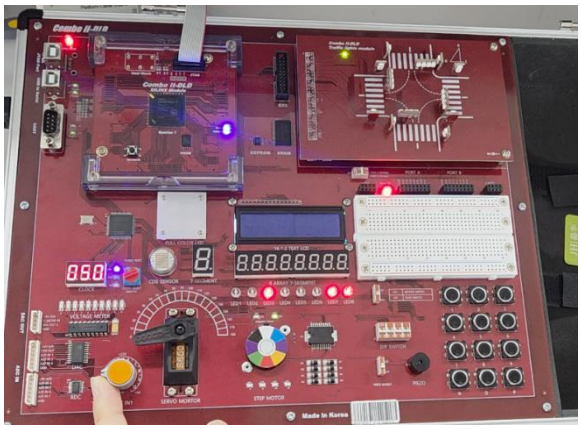


그림 29: 실습3 FPGA 보드 시연 사진

링크: FPGA 실습3 보드 시연 영상

## 9. 문제 발생 및 해결 과정

### ADC 보드 동작과 채널 선택 문제

ADC를 처음 보드에 적용했을 때 LED가 아날로그 입력 변화에 대해 일정하게 반응하지 않는 것으로 보였고, 000에서 조도가 반응하고 001에서 가변저항이 반응하는 등 선택 채널도 예상과 뒤바뀌었다. 이 단계에서는 XDC의 핀 배치, 명령에 포함되는 채널값, 직렬 데이터의 송수신 시점을 차례로 검토하였다.

serial_frame16.v에서는 ADC의 출력 비트를 직렬 클록이 내려가기 전에 저장하고 다음 송신 비트를 미리 준비하도록 수정하였다. 프레임 사이의 짧은 대기 구간도 추가하였다. 이어서 adc.v에서 명령 생성에 전달하는 선택값을 add_sel ^ 3'b001로 변경하여 실제 보드의 000·001 대응을 맞췄다. 두 파일을 반영해 새 비트스트림으로 재시연한 결과 가변저항과 조도가 각 선택 상태에 맞게 반응하였다.

## 10. 예상 값 · Vivado · 보드 결과 비교

| 실험 | 예상 및 판정 기준 | 소프트웨어 확인 | 실제보드 확인 |
|---|---|---|---|
| 과제3 | 버튼마다 PA5 LED 간격 1-2-4-8-1초 | 빌드 완료 | 내장 LED 시연 |
| 과제4 | PB4 PWM의 듀티비 25-50-75-25% | 빌드 완료 | 오실로스코프 측정 |
| 실습1 | 8개의 단일 키의 음, 무음 리셋 | TB PASS, 파형 확인 | 버튼, 피에조 시연 |
| 실습2 | 4방향 순환과 보행자, 비상 점멸 | TB PASS, 파형 확인 | 확장 모듈 사진, 영상 |
| 실습3 | 채널 선택 및 수신 값의 LED 표시 | TB PASS, Vivado 구현 | 가변저항 및 조도 시연 |

STM32의 빌드 성공은 프로그램이 컴파일되었다는 결과이며, 시간 간격과 PWM의 실제 출력은 보드 시연으로 확인하였다. FPGA의 테스트벤치는 정해진 디지털 입력 조건을 검사한다.

## 11. Bitstream 해시 표

| 실습 | Bitstream | SHA-256 |
|---|---|---|
| 피아노 | Lab5_piano.bit | 4AE6254... |
| 신호등 | Lab5_traffic.bit | F1C3571... |
| ADC | Lab5_adc.bit | B90FA08... |

링크: 세 비트스트림 파일

## 12. 결론

STM32 과제3에서는 버튼 외부 인터럽트로 선택 간격을 변경하고 TIM6 주기 인터럽트로 PA5 LED의 상태 변경 시점을 결정하였다. 과제4에서는 TIM3 PWM의 비교값을 바꾸어 PB4 파형의 듀티비를 조절하였다. FPGA의 피아노·신호등은 테스트벤치 검사와 Vivado 구현을 거쳐 보드에서 동작을 시연하였다. ADC에서는 사전 시뮬레이션 이후 실제 보드에서 채널 대응과 표시값 문제를 발견하였고, 직렬 송수신 및 채널 선택 코드를 수정한 뒤 재시연하였다. 이를 통해 시뮬레이션, 구현 화면, 실제 장치의 입출력이 각각 확인해 주는 범위를 구분하면서 결과를 판단하였다.

## 참고 문헌

[1] J. Valvano and R. Yerraballi, "Embedded Systems—Shape the World, Chapter 12: Interrupts," §12.5, *Periodic Interrupts*.